

Useful Codes for Exploring scRNA-seq Data

A beginner-friendly guide

By Júlia Onishi Ibe

Table of contents

01 Get the data

02 Prepare the data

03 Visualize the
cells

04 Ask a question

Get the data

1. Install and load packages in RStudio

```
install.packages("Seurat")  
install.packages("ggplot2")
```



Run this only once. This installs the packages on your computer.

```
library(Seurat)  
library(ggplot2)
```

```
library(Seurat)  
library(ggplot2)
```



Run this each time you open a new R session. This loads the packages so you can use their functions.

Tip: To run a line or block of code in RStudio, simply place your cursor in the code and press Ctrl + Enter.

Get the data

2. Load your data

Single-cell datasets can be provided in different formats. Here, our data are provided as three files containing the expression matrix, genes and cell barcodes.

1

```
library(Seurat)
data_dir <- "C:/Users/Júlia/OneDrive/Documents/GSE296176_ZEBRAFISH"
```

→ This loads your data/directory

2

```
counts <- ReadMtx(
  mtx = file.path(data_dir, "GSM8966473_sample1_matrix.mtx.gz"),
  cells = file.path(data_dir, "GSM8966473_sample1_barcodes.tsv.gz"),
  features = file.path(data_dir, "GSM8966473_sample1_genes.tsv.gz"))
```

→ This loads the expression matrix.
Note: File names may vary between datasets. Make sure that the names used in ReadMtx() exactly match the files you downloaded.

3

```
seurat_obj <- CreateSeuratObject(
  counts = counts,
  project = "ZebrafishHeart_36hpf"
)
```

→ This creates a Seurat object. Your data are now inside R and ready for preprocessing.

Get the data

`dim(counts)`: this function show us the number of genes and cells

```
dim(counts)
```

```
> dim(counts)
[1] 32057 7300
```

genes cells

Prepare the data

1. Quality control: *Are our cells of good quality?*

1

```
mt_genes <- grep(  
  "^mt-",  
  rownames(seurat_obj),  
  value = TRUE,  
  ignore.case = TRUE  
)
```

Here, `grep()` searches the gene names in our Seurat object and identifies those corresponding to mitochondrial genes.

2

```
seurat_obj[["percent.mt"]] <- PercentageFeatureSet(  
  seurat_obj,  
  features = mt_genes  
)
```

Here, we calculate what percentage of each cell's total RNA counts comes from mitochondrial genes.

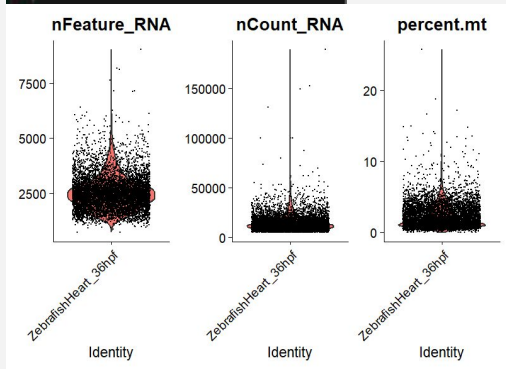
Prepare the data

1. Quality control: *Are our cells of good quality?*

3

```
VlnPlot(  
  seurat_obj,  
  features = c(  
    "nFeature_RNA",  
    "nCount_RNA",  
    "percent.mt"  
  ),  
  ncol = 3  
)
```

Here, we use VlnPlot() to visualize three quality-control metrics for all cells in our dataset. VlnPlot() allows us to visually inspect the quality of our cells before continuing with the analysis.



nFeature_RNA

→ How many different genes were detected in each cell?

nCount_RNA

→ How much RNA was detected in each cell?

percent.mt

→ What proportion of the RNA comes from mitochondrial genes?

Prepare the data

1. Quality control: *Filtering low-quality cells*

Based on these distributions and the characteristics of the dataset, we define appropriate thresholds.

```
seurat_obj <- subset(  
  seurat_obj,  
  subset =  
    nFeature_RNA > 200 &  
    nFeature_RNA < 6000 &  
    percent.mt < 15  
)
```

nFeature_RNA > 200 → removes cells with few detected genes;
nFeature_RNA < 6000 → removes cells with an exceptionally high number of detected genes;
percent.mt < 15 → removes cells with a high proportion of mitochondrial RNA.

Tip: Don't choose a threshold first and force the data to fit it. Look at your data first.

Prepare the data

2. Normalization

NormalizeData() adjusts for differences in sequencing depth between cells, making gene expression levels more comparable across cells.

```
seurat_obj <- NormalizeData(  
  seurat_obj,  
  normalization.method = "LogNormalize",  
  scale.factor = 10000  
)
```

In simple terms: the expression counts are normalized according to the total number of transcripts detected in each cell and then log-transformed. This reduces technical variation related to differences in sequencing depth while preserving biologically meaningful differences in gene expression.

Prepare the data

3. Identification of Highly Variable Features

After normalization, we identify genes that show high variability across cells. These highly variable genes are more informative for downstream analyses such as PCA and clustering.

```
seurat_obj <- FindVariableFeatures(  
  seurat_obj,  
  selection.method = "vst",  
  nfeatures = 2000  
)  
VariableFeaturePlot(seurat_obj)
```

→ You can then visualize the most variable genes

Tip: nfeatures is a user-defined parameter. Common values range from **2,000 to 3,000 highly variable genes**.

Prepare the data

4. Scaling the Data

Before PCA, the expression values are scaled so that genes with different expression ranges can contribute more equally to the analysis.

```
seurat_obj <- ScaleData(seurat_obj)
```

This prevents highly expressed genes from dominating downstream analyses simply because of their larger numerical values.

Tip: Scaling is particularly important before dimensionality reduction methods such as **PCA**.

Prepare the data

5. Principal Component Analysis (PCA)

PCA is a **dimensionality reduction method** that summarizes the main sources of variation across cells using a smaller number of dimensions called **principal components (PCs)**.

```
seurat_obj <- RunPCA(  
  seurat_obj,  
  features = VariableFeatures(seurat_obj)  
)
```

In simple terms: PCA transforms the expression of thousands of genes into a smaller set of PCs that capture the major patterns of variation between cells.

Tip: PCA is performed using the **highly variable genes** identified in the previous step, rather than all genes in the dataset.

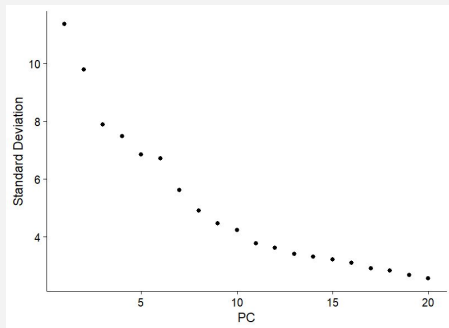
Prepare the data

6. Choosing the Number of PCs

The **Elbow Plot** helps us determine how many principal components should be retained for downstream analyses.

```
ElbowPlot(seurat_obj)
```

The plot shows the **standard deviation of each PC**. PCs at the beginning capture more variation, while later PCs contribute progressively less. In this dataset, the curve begins to flatten around **PC 10–12**, suggesting that approximately **10–12 PCs** may be sufficient to capture the major sources of variation. We will use PCs 1–10 because the elbow is at PC 10.



Tip: The Elbow Plot is a useful guide, but there is **no universal cutoff**. The number of PCs should be evaluated together with the biological context and downstream clustering results.

Prepare the data

7. Finding Neighbors

After selecting the principal components, we use them to identify **cells that are similar to each other** based on their gene expression profiles.

```
seurat_obj <- FindNeighbors(  
  seurat_obj,  
  dims = 1:10  
)
```

Here, `dims = 1:10` means that the **first 10 PCs** are used to calculate cell-to-cell similarity.

In simple terms: `FindNeighbors()` builds a network in which each cell is connected to other cells with similar expression profiles. This network is then used for **clustering**.

Tip: The number of PCs used here should match the PCs selected during the previous step.

Prepare the data

8. Clustering Cells

Cells are grouped into **clusters** based on similarities in their gene expression profiles. The **resolution** parameter controls the **granularity of clustering**. Higher values generally produce more clusters, while lower values produce fewer, broader clusters.

```
seurat_obj <- FindClusters(  
  seurat_obj,  
  resolution = 0.5  
)
```

In simple terms: cells that are more similar to each other are grouped into the same cluster.

Tip: resolution is a user-defined parameter. There is no single optimal value; it should be evaluated according to the dataset and the biological question.

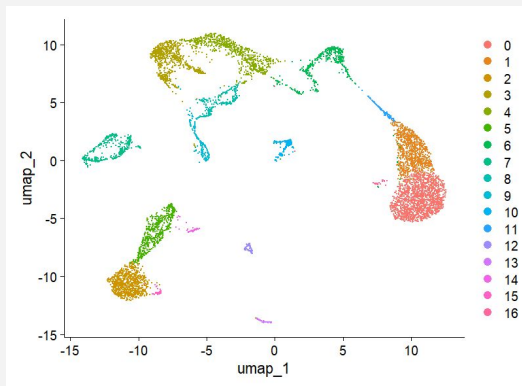
Visualize

UMAP (Uniform Manifold Approximation and Projection) is a dimensionality reduction method used to visualize high-dimensional single-cell data in **two dimensions**.

```
seurat_obj <- RunUMAP(  
  seurat_obj,  
  dims = 1:10  
)
```

```
DimPlot(  
  seurat_obj,  
  reduction = "umap"  
)
```

Each point represents a **cell**, and cells that are closer together have more similar expression profiles. The colors represent the clusters identified in the previous step.



Tip: UMAP is primarily a **visualization tool**. The apparent distance between clusters in a 2D UMAP should not be interpreted as a direct measure of biological distance.

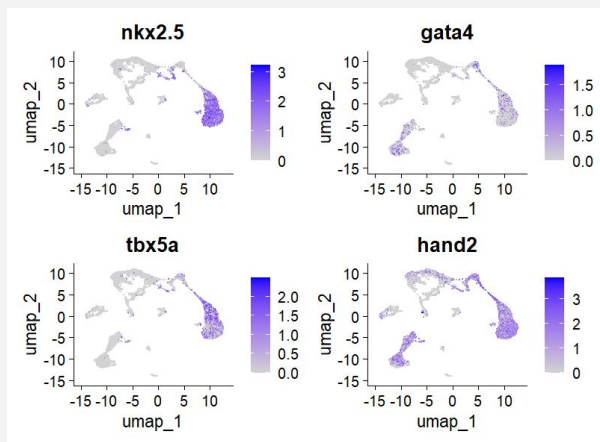
Ask a question

Example: Where are cardiac developmental genes expressed?

Genes of interest:

- **Nkx2-5** - cardiac progenitor specification and cardiomyocyte differentiation
- **Gata4** - cardiac morphogenesis and myocardial development
- **Tbx5** - cardiac chamber and conduction system development
- **Hand2** - cardiac morphogenesis, particularly ventricular development

```
FeaturePlot(  
  seurat_obj,  
  features = c(  
    "nkx2.5",  
    "gata4",  
    "tbx5a",  
    "hand2"  
  ),  
  ncol = 2  
)
```



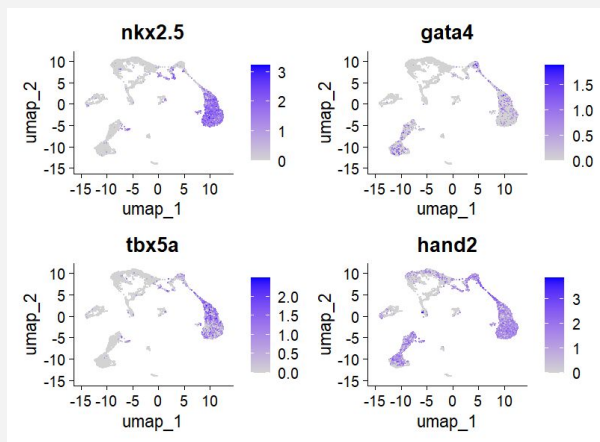
Ask a question

Example: Where are cardiac developmental genes expressed?

Genes of interest:

- **Nkx2-5** - cardiac progenitor specification and cardiomyocyte differentiation
- **Gata4** - cardiac morphogenesis and myocardial development
- **Tbx5** - cardiac chamber and conduction system development
- **Hand2** - cardiac morphogenesis, particularly ventricular development

```
FeaturePlot(  
  seurat_obj,  
  features = c(  
    "nkx2.5",  
    "gata4",  
    "tbx5a",  
    "hand2"  
  ),  
  ncol = 2  
)
```



Tip: Gene names are **organism-specific**. Always check the gene naming convention and confirm that your genes of interest are present in the dataset before performing downstream analyses.

Ask a question

Example: What genes characterize this cluster?

FindMarkers() identifies genes that are differentially expressed in one cell population compared with another. **In simple terms:** this analysis identifies genes that are expressed at higher levels in cluster 0, for example, compared with the remaining cells.

Marker genes can help us **interpret and annotate cell clusters** by comparing their expression profiles with known cell-type markers.

```
# Identify genes differentially expressed
# in cluster 0.

markers_cluster0 <- FindMarkers(
  seurat_obj,
  ident.1 = 0
)

# Display the top 10 genes ranked by average log2 fold change.

head(
  markers_cluster0[
    order(
      markers_cluster0$avg_log2FC,
      decreasing = TRUE
    ),
    1,
    10
  ]
)
```

	p_val	avg_log2FC	pct.1	pct.2	p_val_adj
hspb11	4.034970e-292	5.180664	0.302	0.020	1.293490e-287
tor1l3	1.270324e-13	5.083417	0.010	0.000	4.072279e-09
ostn	8.499244e-29	4.995130	0.026	0.001	2.724603e-24
tpma	0.000000e+00	4.985865	0.403	0.023	0.000000e+00
avpr1aa	2.255242e-62	4.915598	0.057	0.002	7.229630e-58
tnni2a.4	0.000000e+00	4.834627	0.721	0.051	0.000000e+00
tnni4a	2.201694e-159	4.822381	0.178	0.013	7.057971e-155
tnni2b.1	0.000000e+00	4.785983	0.577	0.042	0.000000e+00
fitm1l	5.238723e-39	4.591399	0.035	0.001	1.679377e-34
myh7l	0.000000e+00	4.494484	0.977	0.321	0.000000e+00

Tip: A marker gene is not necessarily a gene that is highly expressed overall. It is a gene that is more characteristic of one population compared with others.

From Quick Start to Biological Questions

This tutorial provides the **first steps toward exploring single-cell RNA-seq data**.

After completing this workflow, you can already:

- **Visualize gene expression** across cell populations
- **Explore and interpret cell clusters**
- Use known **marker genes to annotate cell types**
- Begin asking **biological questions** about cellular composition and gene expression
- Build a foundation for more advanced scRNA-seq analyses

For developmental biology, this means moving from “what cells are present?” to “how do cells acquire and change their identities during development?”

Have fun exploring your cells and happy clustering! 

Thank you!

Contact: j295880@dac.unicamp.br